Workshop 3 Systems Analysis and Desing

Jose Luis Leudo Mosquera (20251020148),
David Santiago Rojas Hernandez (20251020153),
Adrian Mauricio Hernandez Ortiz (20251020147),
Johan Sebastian Candia Marin (20251020139)
Dept. of Systems Engineering
Universidad Distrital Francisco José de Caldas
Bogotá D.C., Colombia
Advisor: Carlos Andrés Sierra, M.Sc

***Abstract*—resumen**

***Index Terms*—palabras claves**

I. System Design Document

A. Executive Summary

The Community Education Volunteer Coordination Platform is a digital system designed to efficiently connect volunteers with tutoring and educational support projects in vulnerable communities of Bogotá D.C. This document consolidates the evolution of the project from the initial analysis (Workshop 1) through the detailed technical design (Workshop 2), now incorporating principles of robust engineering, comprehensive risk management, and formal project planning.

1) Design Evolution: Throughout the three workshops, the project has matured significantly:

- **Workshop 1:** Problem identification, primary data collection via survey of 50 potential UD volunteers, emergent behavior analysis, and detection of bottlenecks in the manual assignment process.
- **Workshop 2:** Formalization of 8 functional and 5 non-functional requirements, three-tier architecture design, definition of the weighted matching algorithm $C(v, p)$ with four compatibility dimensions, and a 4-phase implementation strategy.
- **Workshop 3 (present):** Robust architectural refinement with quality standards (ISO 9001), exhaustive risk analysis (ISO 31000), formal project management plan, and implementation readiness assessment.

2) Current System Status:

TABLE I
Key Project Indicators

Indicator	Status	Detail
Functional requirements	Defined	8 FRs documented and prioritized
Non-functional requirements	Defined	5 NFRs with objective metrics
System architecture	Designed	3-tier architecture + microservices
Matching algorithm	Specified	$C(v, p)$ formula with survey-validated weights
Field data	Collected	50 potential volunteers surveyed

TABLE II
Applied Engineering Principles

Principle	Standard	Application in the System
Modularity	ISO 9001:2015	Separation into layers: Presentation, Application, Data. Each module is independently deployable.
Fault Tolerance	ISO 9001:2015	Waiting queue for assignments during volunteer shortage; offline mode for sessions.
Scalability	CMMI-DEV	Architecture ready for 80+ concurrent users; Google Sheets API scalable to relational database.
Maintainability	ISO 9001:2015	Modular design compatible with microservices; module updates without affecting the global system.
Usability	ISO 9001:2015	Registration in under 5 minutes without prior training; interface adapted to low connectivity.
Traceability	PMBOK 6th ed.	Complete logging of sessions, assignments, and academic impact metrics.

B. Robust Architecture Design

1) Applied Engineering Principles: The architectural refinement from Workshop 2 incorporates the following robust engineering principles, aligned with international standards:

2) System Architecture — Detailed Description: The three-tier architecture defined in Workshop 2 is refined with the following additional components to guarantee robustness.

a) Presentation Layer (Frontend):

- **Volunteer Portal:** Profile registration, view of assigned projects, session confirmation, and hour tracking. Designed to function on devices with limited connectivity.
- **Institution Portal:** Publication of academic projects, specification of needs by area and level, and access to student progress reports.

- **Administration Panel:** Global supervision, assignment management, matching algorithm configuration, and impact report generation.
- **Reports & Analytics Module:** Visualization of key indicators: attendance rate, academic improvement, volunteered hours, and assignment satisfaction.

b) Application Layer (Backend):

- **Volunteer Registration Module:** Capture and validation of academic profile, teaching experience, schedule availability, and geographic location.
- **Institution Registration Module:** Management of educational projects, definition of academic needs by subject and school level.
- **Matching Algorithm Engine (system core):** Implements the formula $C(v,p) = 0.40 \cdot Skill + 0.30 \cdot Schedule + 0.15 \cdot Location + 0.15 \cdot Experience$. Generates compatibility rankings and executes automatic or assisted assignments.
- **Scheduling Engine:** Real-time conflict detection before confirming assignments; support for multiple modalities: in-person, virtual, and hybrid.
- **Academic Impact Monitor:** Logging of completed sessions, capture of academic improvement metrics, and generation of volunteering effectiveness indicators.
- **Notification Service:** Sends confirmations, reminders, and cancellations via email; extensible to SMS.

c) Data Layer: The system persists its state in four specialized repositories:

- **Volunteer Profile DB:** Academic competencies, session history, performance evaluations.
- **Projects & Schedules DB:** Active projects per institution, academic needs, slot availability.
- **Sessions DB:** Log of completed sessions, attendance, tutor observations.
- **Analytics Repository:** Aggregated data for reports and academic impact indicators.

Technical Decision — Database: Given the initial project scope, Google Sheets API will be implemented as a temporary database, enabling read/write from Python. This decision is aligned with the available technological infrastructure and team capacity. Migration to PostgreSQL is planned for Phase 3, when the system reaches 50+ active users.

3) Refined Architecture — Interaction Flows:

TABLE III
SYSTEM INTERACTION FLOWS

Flow	Origin	Destination	Description
F1	Volunteer Portal	Registration Module	Profile submission with skills, availability, and location
F2	Institution Portal	Institution Module	Project publication with academic needs
F3	Matching Engine	Profile DB + Project DB	Data query to calculate $C(v,p)$
F4	Matching Engine	Scheduling Engine	Availability validation before assignment
F5	Scheduling Engine	Notification Service	Confirmation sent to volunteer and institution
F6	Completed Session	Impact Monitor	Attendance and academic metrics logging
F7	Impact Monitor	Analytics Repository	Storage of indicators for reports
F8	Admin Panel	Matching Engine	Manual adjustment of $C(v,p)$ algorithm weights

TABLE IV
RISK ASSESSMENT SCALE

Probability	Description	Impact	Description
High (3)	Expected occurrence in the project	High (3)	Affects the main system objective
Medium (2)	Possible but not expected	Medium (2)	Significant delays or partial degradation
Low (1)	Unlikely	Low (1)	Minor impact, easily recoverable

C. Risk Management Plan

This risk management plan applies the PMBOK 7th edition and ISO 9001:2015 framework for the identification, assessment, and mitigation of risks in the development and operation of the platform.

1) Risk Assessment Scale:

2) Technical Risk Register:

3) Operational Risk Register:

4) Contingency Plan for Critical Risks: **RT01 / RO01 — Critical Risks (Priority 9):**

RT01 — Low-quality data: If more than 20% of profiles have incomplete critical fields after 2 weeks post-launch, a profile completion campaign will be activated with incentives ("Verified Volunteer" badge) and the administrator will perform manual review of priority profiles.

RO01 — Volunteer shortage: If project coverage falls below 70%, a dissemination protocol will be activated through UD academic networks, inactive volunteers will be notified, and the system will prioritize critical projects (highest impact on school dropout).

D. Quality Assurance Framework

The quality assurance framework is aligned with ISO 9001:2015 for development process quality management.

1) Quality Metrics and Acceptance Criteria:

2) Testing Strategy:

a) Unit Tests:

- Minimum 80% code coverage of the $C(v,p)$ algorithm with test cases for the four components: Skill, Schedule, Location, Experience.

TABLE V
TECHNICAL RISK REGISTER

ID	Risk	P	I	Mitigation Strategy
RT01	Poor quality of data entered by volunteers and institutions affects $C(v,p)$ accuracy	3	3	Mandatory validation of critical fields with predefined formats. Descriptive error messages. Periodic review of incomplete profiles.
RT02	Google Sheets API service failure (quota exceeded or outage)	2	3	Implement local cache for frequent data. Exception handling with automatic retries. Documented PostgreSQL migration plan.
RT03	Low connectivity in vulnerable communities prevents platform use	3	2	Offline mode for session attendance. Automatic sync on reconnection. Lightweight PWA interface.
RT04	Algorithm produces incompatible assignments due to incorrectly calibrated weights	2	3	Exhaustive unit testing of $C(v,p)$ module. Simulation with real survey data. Manual adjustment available via admin panel.
RT05	Undetected scheduling conflicts generate double volunteer assignments	2	2	Conflict validation before confirmation (Scheduling Engine). Real-time slot blocking.

TABLE VI
OPERATIONAL RISK REGISTER

ID	Risk	P	I	Mitigation Strategy
RO01	Fluctuation in available volunteer supply prevents covering all projects	3	3	Waiting list with temporary assignments. Proactive notifications to volunteers with open slots. Weekly coverage reports for the administrator.
RO02	Low platform adoption by educational institutions	2	3	Training and onboarding sessions for institutions. Simplified user documentation. Pilot with 2–3 partner institutions before general launch.
RO03	High session cancellation rate by volunteers	2	2	Automatic reminders (48h and 2h before). Reliability indicator on volunteer profile. Cancellation policy with 24-hour minimum window.
RO04	Changes in institution academic calendars affect scheduled availability	2	2	Scheduling engine with mass rescheduling function. Automatic alerts to affected volunteers. Monthly and weekly demand reports.

TABLE VII
QUALITY METRICS AND ACCEPTANCE CRITERIA

Characteristic (ISO 25010)	Metric	Target	Validation Method
Performance Efficiency	Matching algorithm response time	<60 s	JMeter load test, 100 concurrent users
Reliability / Availability	Monthly system uptime	>99%	Continuous monitoring with automatic alerts
Usability	Volunteer registration completion time	<5 min	Usability tests with 10 UD users
Functional Accuracy	Assignment satisfaction	>80% positive	Post-assignment survey (Likert 1–5)
Maintainability	Module update impact	0 unrelated modules affected	Regression tests after each update
Scalability	Concurrent users without degradation	80+	Load test with real-scenario simulation
Social Impact	School dropout trend in pilot institutions	Decreasing in 6 months	Pre/post academic indicator comparison

- Validation of edge cases: volunteer with zero availability, project with no candidates, tied scores.
- Framework: pytest (Python) or JUnit (Java) depending on final implementation.

b) Integration Tests:

- Full flow validation: Registration → Matching → Assignment → Notification → Session logging.
- Integration tests with Google Sheets API: write, read, and error handling.
- Verification of scheduling conflict detection in multiple assignment scenarios.

c) Load Tests:

- Simulation of 100 concurrent users accessing the Volunteer Portal and Institution Portal simultaneously.
- Matching algorithm load test with 500 volunteer-project pairs.

d) Usability Tests:

- Pilot sessions with 10 UD students unfamiliar with the system. Task: complete registration and view assigned project.
- Metrics: completion time, number of errors, satisfaction (SUS — System Usability Scale).

3) Continuous Improvement Process: The PDCA cycle applied to the system:

- **Plan:** Define quality metrics per sprint. Review post-assignment satisfaction survey results.
- **Do:** Implement identified improvements. Adjust $C(v,p)$ algorithm weights based on user feedback.
- **Check:** Compare current metrics against acceptance criteria. Biweekly quality reports to the team.

- **Act:** Document lessons learned. Scale validated improvements to the next development cycle.

E. Design Evolution and Lessons Learned

1) Integrated Feedback from Workshop 1: The analysis of 50 surveyed volunteers in Workshop 1 revealed insights that directly shaped the design:

- **Finding:** 54% of volunteers have fewer than 2 hours available per week. *Impact:* The system prioritizes short sessions (45–60 min) and the algorithm does not penalize reduced availability if concentrated in high-demand time slots.
- **Finding:** Engineering (40%) and Pedagogy (22%) dominate the volunteer profile. *Impact:* $Skill(v, p)$ weights favor assignments in mathematics, technology, and literacy where qualified supply is highest.
- **Finding:** Virtual and in-person modalities tie at 36% preference. *Impact:* The $Location(v, p)$ module evaluates modality separately: geographic score for in-person, connection quality assessment for virtual.
- **Bottleneck detected:** The manual assignment process depended on the quality of entered data. *Impact:* Mandatory validation of critical fields (P2, P5, P6, P7) was added to the registration module.

2) Integrated Feedback from Workshop 2:

- **3-tier architecture:** Refined with explicit separation of responsibilities and definition of inter-module interfaces, reducing coupling.
- **Algorithm $C(v, p)$:** Weights were qualitatively validated against survey results. Quantitative calibration will be performed in Phase 2 of implementation with real data.
- **Risk management:** Sensitivity factors identified in Workshop 1 (volunteer variability, connectivity, scheduling conflicts) were formalized as risks with specific mitigation strategies in this document.

F. References

[1] [2] [3] [4] [5] [6]

II. Project Management Documentation

A. Project Charter

1) Scope Definition: The Community Education Volunteer Coordination Platform is a digital system designed to connect university volunteers with educational institutions serving vulnerable communities in Bogotá D.C. Its primary function is to automate and optimize the assignment of tutors based on academic skills, schedule availability, geographic location, and pedagogical experience.

2) Project Objectives:

- Reduce volunteer-to-project assignment time through an automated weighted compatibility algorithm.
- Improve match quality between volunteer skills and institutional academic needs.
- Ensure conflict-free schedule coordination and session confirmation workflows.
- Measure and report educational impact through quantitative indicators: attendance rate, academic improvement, and volunteer hours.
- Provide a scalable and maintainable platform capable of supporting 80 or more concurrent users.

TABLE VIII
Stakeholder Summary

Stakeholder	Role	Interest / Expectation
University Volunteers	Primary Users	Flexible schedule, clear assignments, meaningful social impact
Educational Institutions	Beneficiaries / Clients	Reliable academic support tailored to student needs
Students (K-12)	End Beneficiaries	Consistent tutoring that improves academic performance
Project Team (UD)	Developers / Analysts	Deliver a functional, tested prototype aligned with requirements
Prof. C.A. Sierra	Academic Advisor	Methodological rigor, proper documentation, system viability
Universidad Distrital	Institutional Sponsor	Social impact, alignment with the community service mission

3) Stakeholders:

4) Success Criteria:

- Matching algorithm response time under 1 minute for any volunteer-project pair.
- System availability above 99% monthly uptime.
- Assignment accuracy with more than 80% positive feedback from volunteers and institutions.
- Volunteer registration completed in under 5 minutes without prior training.
- Measurable downward trend in school dropout rates within partner institutions.

B. Team Structure and Roles

1) Organizational Chart: The project team is organized into four primary roles under the direction of the academic advisor. Each member leads a functional area while participating in cross-functional project decisions.

2) Responsibility Assignment Matrix (RACI):

C. Project Schedule

1) Implementation Phases Overview: The project follows a four-phase roadmap derived from the implementation strategy defined in Workshop 2. Each phase builds on the deliverables of the previous one, with explicit milestones, dependencies, and critical path checkpoints.

2) Milestones and Critical Path:

TABLE IX
TEAM ROLES AND RESPONSIBILITIES

Team Member	Primary Role	Responsibilities
J.L. Leudo Mosquera	Systems Architect / Backend Lead	Three-tier architecture; Google Sheets API integration; matching algorithm
D.S. Rojas Hernandez	Algorithm Engineer / QA Lead	Compatibility algorithm $C(v, p)$; unit and integration testing; performance metrics
A.M. Hernandez Ortiz	Frontend Dev / Documentation Lead	Volunteer and institution portals; reports module; technical documentation
J.S. Candia Marin	Data Analyst / Integration Lead	Survey data analysis; database schema; scheduling engine; CI/CD setup
Prof. C.A. Sierra	Academic Advisor	Methodological guidance; milestone review; final evaluation

TABLE X
RACI MATRIX (R=RESPONSIBLE, A=ACCOUNTABLE, C=CONSULTED, I=INFORMED)

Activity	Leudo	Rojas	Hdz.	Candia	Adv.
Requirements Analysis	A	R	R	C	I
Architecture Design	R	C	C	I	I
Matching Algorithm	C	R	C	R	I
DB Schema / Sheets	I	C	C	R	I
Frontend Portals	C	I	R	C	I
Testing & QA	C	R	I	C	I
Documentation	I	C	R	C	I
CI/CD Pipeline	C	I	I	R	I

TABLE XI
IMPLEMENTATION PHASES

Ph.	Name	Key Deliverables	Duration
1	Foundations	GitHub repo, CI/CD pipeline, registration modules, Google Sheets schema	Weeks 1–3
2	Matchmaking Engine	$C(v, p)$ algorithm, unit tests, DB integration, matching endpoint	Weeks 4–6
3	Scheduling & Notif.	Conflict-free scheduling, session confirmation, email service	Weeks 7–8
4	Impact Monitoring	Session forms, impact dashboard, automated reports	Weeks 9–10

3) Critical Path Analysis: The critical path runs sequentially through all four phases: Registration Modules → Matching Algorithm → Scheduling Engine → Impact Reports. Any delay in Phase 1 or Phase 2 directly postpones the entire schedule. The email notification service (Phase 3, optional) sits off the critical path and can be deferred without blocking Phase 4.

TABLE XII
MILESTONES AND CRITICAL PATH

Milestone	Week	Critical?
Repository and CI/CD pipeline operational	1	Yes
Registration modules deployed with Sheets API	3	Yes
Algorithm passes unit tests ($\geq$95% coverage)	5	Yes
Matching endpoint live ($<$1 min, 80 users)	6	Yes
Conflict-free scheduling engine (0 double-bookings)	7	Yes
Email notifications integrated ($<$2 min)	8	No
Impact metrics dashboard complete	10	Yes
Final report and documentation delivered	10	Yes

D. Resource Management Plan

1) Budget Considerations: Given the academic nature of the project and the use of open-source tools and free-tier cloud services, the estimated monetary cost is near zero. The primary resource is team time.

TABLE XIII
RESOURCE SUMMARY

Category	Tool / Service	Notes
Version Control & CI/CD	GitHub (Free Tier)	Public repo
Database (Phases 1–2)	Google Sheets API	Free quota sufficient
Backend Runtime	Python 3.x	Open source
Frontend Framework	HTML/CSS/JS or React	Open source
Dev. Hours (4 members × 10 wks)	Human capital	Primary resource

TABLE XIV
RESOURCE ALLOCATION (HOURS PER MEMBER)

Phase	Leudo	Rojas	Hdz.	Candia
Ph.1 – Foundations	25	15	15	25
Ph.2 – Algorithm	20	30	15	15
Ph.3 – Scheduling	15	20	20	25
Ph.4 – Reporting	15	15	30	15
Total	**75**	**80**	**80**	**80**

2) Resource Allocation by Phase:

3) Capacity Planning:

- Each team member is expected to contribute approximately 8–10 hours per week to the project.
- Weekly sprint planning meetings are held to review progress and re-allocate tasks based on availability and blockers.
- The Google Sheets API free quota (300 requests/minute) is sufficient for a pilot of up to 100 volunteers; migration to PostgreSQL is planned for the post-prototype stage.

E. Communication and Control Plan

1) Stakeholder Communication Strategies:

TABLE XV
COMMUNICATION PLAN

Audience	Medium & Frequency	Content
Internal Team	WhatsApp + GitHub Issues (daily)	Task updates, blockers, quick decisions
Academic Advisor	Written report + presentation (bi-weekly)	Progress review, milestone validation, feedback
Test Users (UD)	Google Forms + WhatsApp (per survey round)	Survey distribution, prototype feedback
Educational Institutions	Email + platform demo (Phase 4)	System walkthrough, usage instructions, impact data

2) Project Monitoring and Control Approaches: The team employs the following control mechanisms to track progress, manage deviations, and ensure quality throughout the project lifecycle:

- **GitHub Issues & Pull Requests:** All development tasks are tracked as issues. Pull requests require peer review before merging to main, ensuring code quality and knowledge sharing.
- **Weekly Sprint Reviews:** The team reviews completed tasks against the milestone plan, identifies slippage, and adjusts workload for the following sprint.
- **CI/CD Pipeline Monitoring:** Automated builds and test suites run on every push. Failing builds trigger immediate team alerts, preventing broken code from reaching the integration branch.
- **Performance Benchmarking:** The matching algorithm is benchmarked against the sub-1-minute response time target. Load tests with 80+ concurrent users are executed before each phase delivery.
- **Risk Register Reviews:** The risk register is reviewed bi-weekly. If a sensitivity factor escalates, the corresponding mitigation mechanism is activated immediately.
- **Advisor Checkpoints:** Formal written reports are submitted at each workshop deadline. Advisor feedback is incorporated in the next sprint.

3) Change Management: Any change to scope, schedule, or resource allocation must: (1) be proposed in writing via a GitHub Issue labelled `change-request`; (2) be reviewed by all four team members within 48 hours; (3) be approved by majority vote and recorded in the project log. Changes affecting the critical path require advisor notification.

F. Risk Register (Summary from Workshop 2)

TABLE XVI
RISK REGISTER SUMMARY

Sensitivity Factor	Like.	Impact	Mitigation
Variability in volunteer supply	High	High	Waiting queue + temporary matching assignments
Scheduling conflicts	Med.	High	Conflict detection engine before every session confirmation
Low connectivity in communities	Med.	Med.	Offline recording mode; data syncs on reconnect
Changes in student academic demand	Med.	Med.	Monthly demand reports; dynamic algorithm weight adjustment
Google Sheets API quota exceeded	Low	High	Migrate to PostgreSQL; quota monitoring alerts
Team member unavailability	Low	Med.	RACI matrix ensures knowledge redundancy across roles

G. Quality Assurance Objectives

1) Performance Metrics Targets:

TABLE XVII
PERFORMANCE METRICS

Metric	Target	Measurement Method
Matching response time	<1 min	Load test, 80 concurrent users
System availability (monthly)	>99%	Uptime monitoring via automated ping
Assignment accuracy (positive feedback)	>80%	Post-session survey to volunteers and institutions
Reduction in school dropout	Downward trend	Pre/post academic period comparison
Volunteer registration time	<5 min	Usability test with UD pilot group
Concurrent user capacity	80+	Stress test simulation

2) Quality Assurance (QA) Strategy:

- **Unit tests:** All functions of the compatibility algorithm $C(v, p)$ and the scheduling engine are covered by unit tests (target $\geq$ 95% code coverage).
- **Integration tests:** End-to-end scenarios covering volunteer registration $\rightarrow$ matching $\rightarrow$ scheduling $\rightarrow$ notification $\rightarrow$ session logging, fully automated.
- **Load testing:** Simulations target 100+ concurrent users without performance degradation, executed prior to each phase delivery.
- **Usability testing:** Pilot sessions with Universidad Distrital students verify that registration is completed in under 5 minutes without assistance.
- **Continuous monitoring:** Automatic alerts notify the team of error rate spikes and response time anomalies through the CI/CD pipeline.

III. Visual Representations and Tools

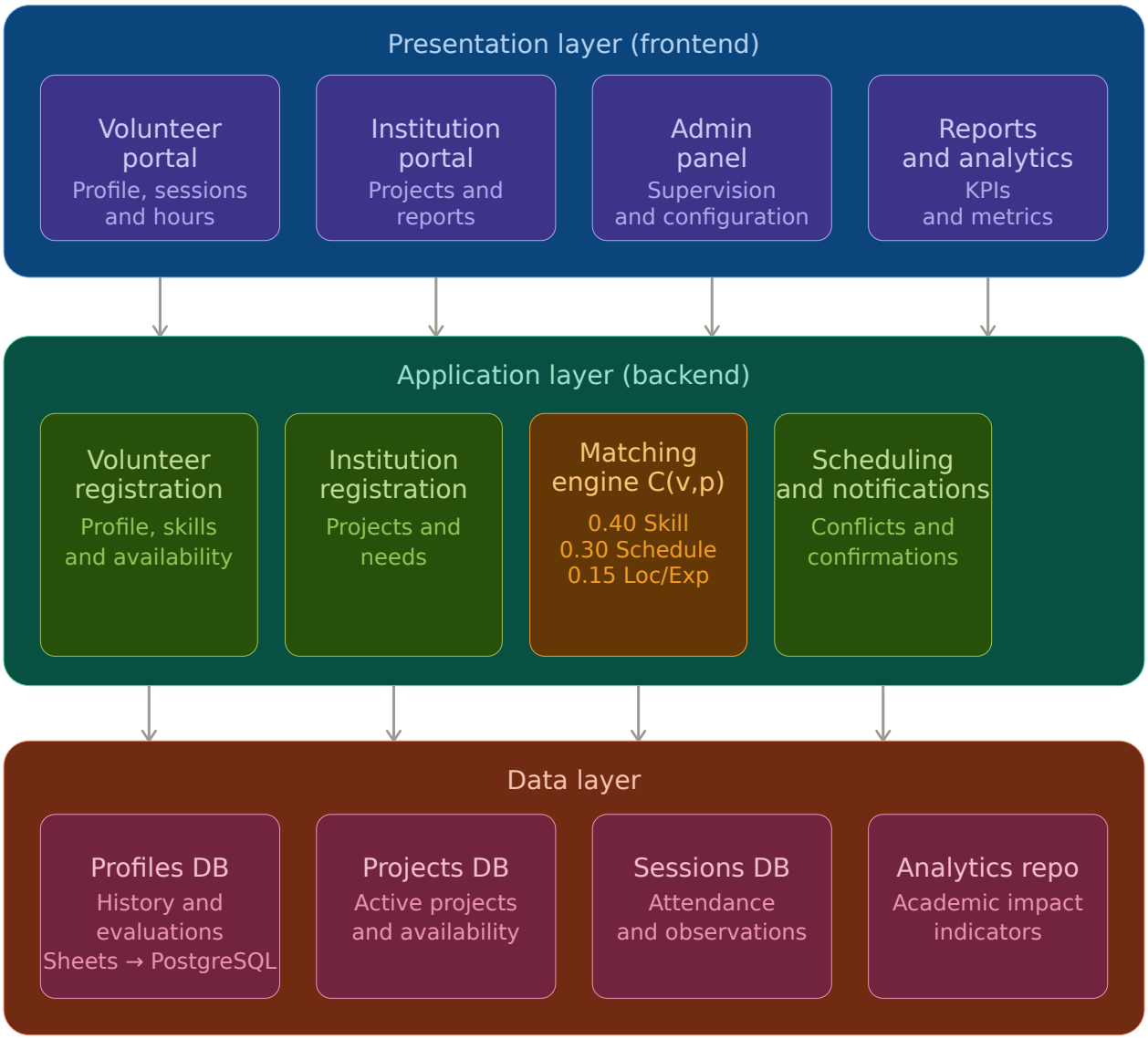

Fig. 1. Robust system architecture — three-tier design (ISO 9001:2015)

Risk management matrix (ISO 31000)			
Prob. / Impact	Low (1)	Medium (2)	High (3)
High (3)	—	RT03 Score 6 — High	RT01, RO01 Score 9 CRITICAL
Medium (2)	—	RT05, RO03 RO04 Score 4 — Medium	RT02, RT04 RO02 Score 6 — High
Low (1)	—	—	—

Critical (9) High (6) Medium (4) Low / None (1–3)

Fig. 2. Risk management matrix (ISO 31000)

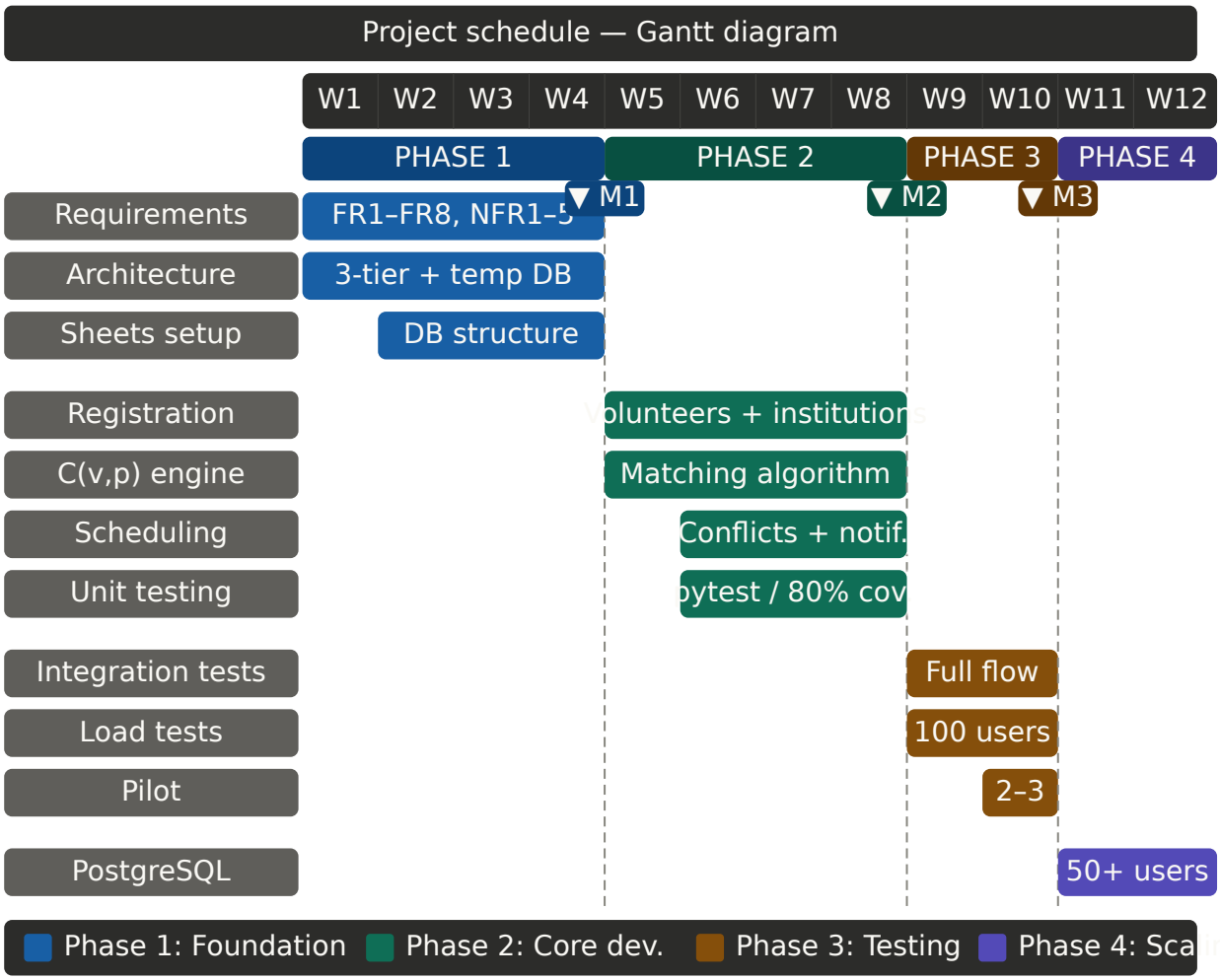

Fig. 3. Project schedule — Gantt diagram

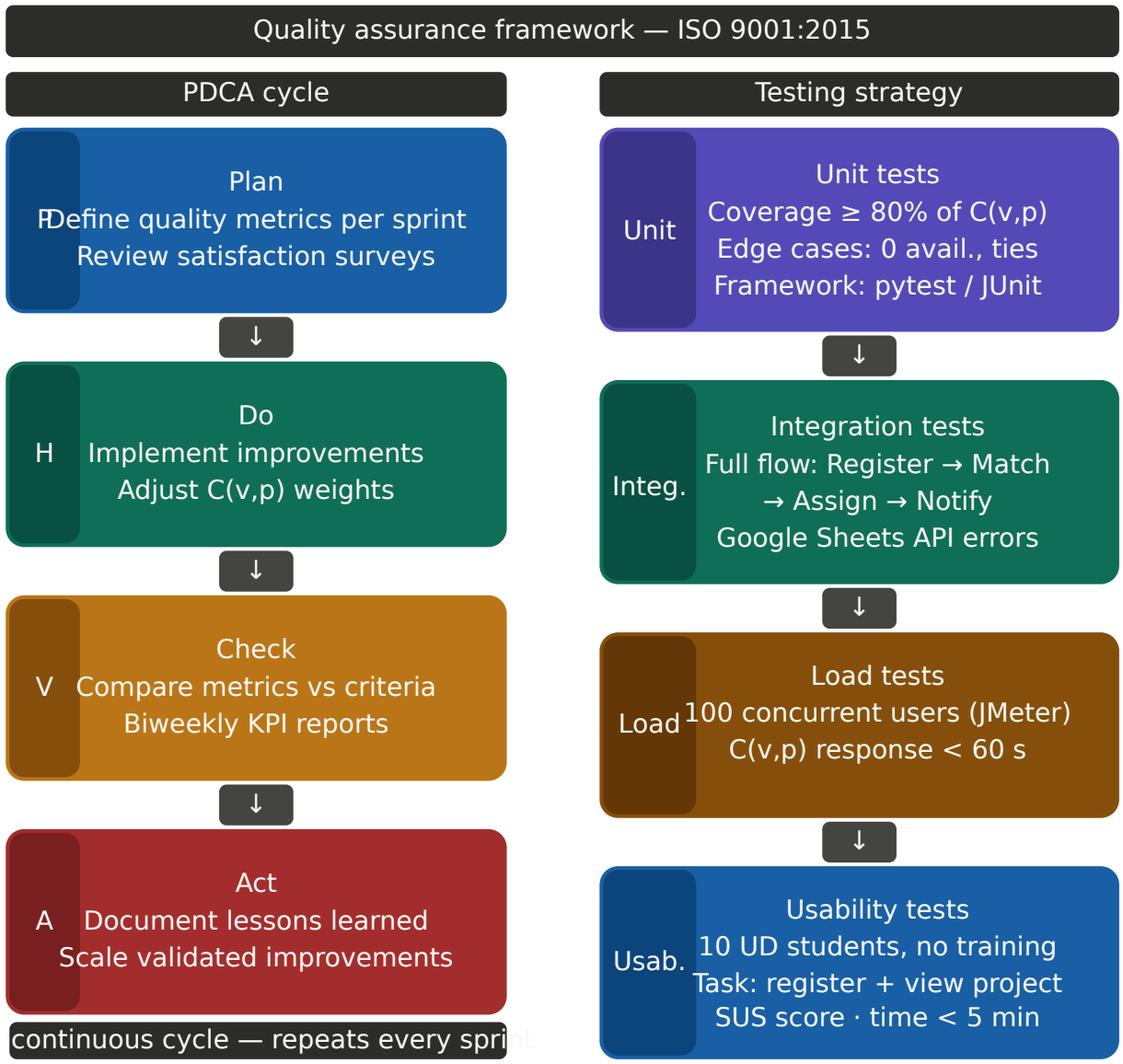

Fig. 4. Quality assurance framework — PDCA cycle (ISO 9001:2015)

References

[1] International Organization for Standardization, "ISO 31000:2018 — Risk Management — Guidelines," ISO, Geneva, Switzerland, International Standard, 2018. [Online]. Available: https://www.iso.org/standard/65694.html

[2] ——, "ISO 9001:2015 — Quality Management Systems — Requirements," ISO, Geneva, Switzerland, International Standard, 2015. [Online]. Available: https://www.iso.org/standard/62085.html

[3] ——, "ISO/IEC 25010:2011 — Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — System and Software Quality Models," ISO, Geneva, Switzerland, International Standard, 2011. [Online]. Available: https://www.iso.org/standard/35733.html

[4] Project Management Institute, *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*, 7th ed. Newtown Square, PA: Project Management Institute, 2021.

[5] CMMI Institute, "CMMI for Development (CMMI-DEV)," https://cmmiinstitute.com/cmmi/dev, 2023, accessed: 2026.

[6] YouTube, "Video reference," https://www.youtube.com/watch?v=fzTV-Ebk5Cg, 2024, accessed: 2026.